Travaux Dirigés d'analyse syntaxique n°9

Licence d'informatique

———

Jeu d'essai - Conflits avec Bison

Ce TP vise à commencer à construire un jeu d'essai pour le projet d'analyseur syntaxique du langage TPC, puis à maitriser les conflits d'analyse syntaxique qui se produisent avec `bison`.

▸ Exercice 1. *Jeu d'essai*

1. *Faites une série de petits programmes TPC corrects, de telle sorte que chaque règle de la grammaire algébrique du projet soit utilisée par au moins un de vos programmes.*
2. *Faites 3 nouveaux programmes TPC corrects obtenus à partir des précédents en faisant de petites modifications autorisées. Inspirez-vous de la définition des lexèmes du langage TPC (sujet du projet, section 2). N'hésitez pas à introduire des possibilités que les développeurs n'utilisent pas.*
3. *Faites un petit programme TPC incorrect avec une seule erreur* ***lexicale****, par exemple un caractère qui apparait là où il est interdit.*
4. *Faites 3 petits programmes TPC incorrects, avec une seule erreur par programme. Insérez uniquement des erreurs* ***syntaxiques****, c'est-à-dire interdites par la grammaire algébrique du projet. Par exemple, la grammaire du projet interdit d'avoir* `===`*, mais elle n'interdit pas la division par zéro, ni d'avoir plusieurs fonctions qui ont le même nom. Si votre projet tourne et que voulez lancer les tests sur votre jeu d'essai, vous pouvez utiliser votre script du TP 3, exercice 3.*

▸ Exercice 2. *Compter les conflits*

Complétez votre programme `flex` *du TP 7, exercice 1.2, et complétez* `instr-tpc.y` *(dans le dossier* `instr`*), pour les coupler ensemble. Pour vos tests, le fichier* `instr.tpc` *contient des instructions conformes à la grammaire de* `instr-tpc.y`*. Combien de conflits empiler/réduire* `bison` *trouve-t-il avec cette grammaire ? Combien de conflits réduire/réduire ?*

▸ Exercice 3. *Résoudre un conflit en modifiant la grammaire*

Lancez la commande `bison` *sur votre* `instr-tpc.y` *avec l'option* `-v` *pour obtenir le fichier texte (extension* `.output`*) qui contient la table d'analyse ascendante. (Vous pouvez ajouter une clause dans votre* `makefile` *pour que la commande* `make table` *lance cette commande, vous en aurez besoin pour gérer les conflits dans votre projet.) Ouvrez le fichier* `.output`*, étudiez les conflits et modifiez la grammaire pour faire disparaitre les conflits sans changer le langage. Reconstituez ce que fait l'analyseur syntaxique avec la solution que vous implémentez.*

▸ **Exercice 4.** ***Implémenter une calculatrice***

Le dossier `expr_arith` *contient un fichier* `expr_arith.lex` *qui définit les lexèmes des expressions arithmétiques, et un fichier* `expr_arith.y` *qui définit une grammaire correspondante, ainsi qu'un Makefile pour les compiler ensemble.*

1. *Compiler les fichiers. Combien y a-t-il de conflits ? Étudier les exemples fournis en ajoutant l'option* `-Wcex` *pour déterminer d'où ils proviennent.*
2. *Ajouter des déclarations dans le fichier* `expr_arith.y` *pour résoudre les conflits, en respectant la priorité des opérateurs :* $*$ *et* $/$ *ont la même priorité, et sont prioritaires sur* $+$ *et* $-$ *qui ont la même priorité. Tous sont associatifs à gauche.*
3. *Ajouter des instructions dans le fichier* `expr_arith.y` *pour que le programme évalue l'expression arithmétique donnée en entrée. Par exemple, sur l'entrée* `1 + 3 * 5` *contenue dans* `test.txt`*, le programme doit afficher* `16`*.*